



# **Dhaka International University**

## Lab Report details:

Lab Report Topic : **Write a program to solve Bankers algorithm using C**

Lab Report No : 04

Course Title : Database Management System

Course Code : 0613-301

| Submitted By:                                             | Submitted To:                                   |
|-----------------------------------------------------------|-------------------------------------------------|
| S M Nabil Ausaf<br>Roll: 01<br>Reg No : CS-D-91-23-124492 | Md. Nazmus Sakib, Lecturer of<br>CSE Department |

Date of Submission:22.11.25

**Lab No: 04**

**Lab Name: Banker's Algorithm**

**Objectives:**

- To understand the concept of deadlock avoidance in operating systems.
- To implement the Banker's Algorithm in C programming language.
- To determine whether a system is in a safe or unsafe state.
- To generate the safe sequence of processes if the system is in safe condition.

**Background Study:**

Deadlock is a situation in which multiple processes wait indefinitely for resources that are held by each other. To avoid deadlock, operating systems use algorithms such as the **Banker's Algorithm**, introduced by Edsger Dijkstra.

The Banker's Algorithm is used to check whether resource allocation to a process leads the system to a safe or unsafe state. The system is declared **safe** only if it can allocate resources to all processes in some order and still avoid deadlock.

This algorithm uses four important matrices:

1. **Allocation Matrix** – Resources currently allocated to each process
2. **Max Matrix** – Maximum resources each process may request
3. **Need Matrix** – Remaining required resources
4. **Available Array** – Currently available resources

The system is safe if it finds a **safe sequence** where all processes can finish.

**Characteristics:**

- Ensures the system never enters a deadlock state.
- A process must declare its maximum resource demand in advance.
- The algorithm checks safety before allocating resources.
- Slightly slower due to continuous safety checks, but reliable.

**Example:**

Consider a system with 3 processes (P0, P1, P2) and 3 resource types.

| Process | Allocation | Max   |
|---------|------------|-------|
| P0      | 1 0        | 7 5 3 |
| P1      | 2 0 0      | 3 2 2 |
| P2      | 3 0 2      | 9 0 2 |

Available resources: **(3 3 2)**

**Algorithm:**

1. Start
2. Input number of processes and resource types
3. Input Allocation matrix, Max matrix, and Available resources
4. Compute Need matrix:
  - $\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$
5. Check safety:
  - If all  $\text{Need} \leq \text{Available}$ , process can execute
  - Release resources after execution
6. If a safe sequence is found  $\rightarrow$  Safe state
7. Else  $\rightarrow$  Unsafe state
8. Stop

### CODE (C Program):

```
here X Banker's Algorithm.c X
1  #include <stdio.h>
2
3  int main() {
4      int n, m;
5      printf("Enter number of processes: ");
6      scanf("%d", &n);
7
8      printf("Enter number of resource types: ");
9      scanf("%d", &m);
10
11     int alloc[n][m], max[n][m], need[n][m], avail[m];
12
13     // Input Allocation Matrix
14     printf("\nEnter Allocation Matrix (%d x %d):\n", n, m);
15     for (int i = 0; i < n; i++)
16         for (int j = 0; j < m; j++)
17             scanf("%d", &alloc[i][j]);
18
19     // Input Max Matrix
20     printf("\nEnter Max Matrix (%d x %d):\n", n, m);
21     for (int i = 0; i < n; i++)
22         for (int j = 0; j < m; j++)
23             scanf("%d", &max[i][j]);
24
25     // Input Available Resources
26     printf("\nEnter Available Resources (%d):\n", m);
27     for (int i = 0; i < m; i++)
28         scanf("%d", &avail[i]);
29
30     // Calculate Need Matrix
31     printf("\nNeed Matrix is:\n");
32     for (int i = 0; i < n; i++) {
33         for (int j = 0; j < m; j++) {
34             need[i][j] = max[i][j] - alloc[i][j];
35             printf("%d ", need[i][j]);
36         }
37         printf("\n");
38     }
```

```

38     }
39
40     int finish[n];
41     for (int i = 0; i < n; i++)
42         finish[i] = 0;
43
44     int safeSeq[n], count = 0;
45
46     // Banker's Algorithm
47     while (count < n) {
48         int found = 0;
49
50         for (int i = 0; i < n; i++) {
51             if (!finish[i]) {
52                 int j;
53                 for (j = 0; j < m; j++) {
54                     if (need[i][j] > avail[j])
55                         break;
56                 }
57
58                 if (j == m) {
59                     for (int k = 0; k < m; k++)
60                         avail[k] += alloc[i][k];
61
62                     safeSeq[count++] = i;
63                     finish[i] = 1;
64                     found = 1;
65                 }
66             }
67         }
68
69         if (!found) {
70             printf("\nSystem is NOT in safe state.\n");
71             return 0;
72         }

```

```

72     }
73 }
74
75 // Print Safe Sequence
76 printf("\nSystem is in SAFE STATE.\nSafe Sequence: ");
77 for (int i = 0; i < n; i++)
78     printf("P%d ", safeSeq[i]);
79
80 printf("\n");
81
82 return 0;
83 }

```

Need Matrix:

7 4 3

1 2 2

6 0 0

System is in SAFE STATE.

Safe Sequence: P1 P3 P0

### Discussion:

In this lab, the Banker's Algorithm was successfully implemented to determine whether a given resource allocation leads the system to a safe or unsafe state. The algorithm computed the Need matrix and then checked each process to see if its remaining requirements could be satisfied with available resources.

If a process could be completed, its allocated resources were released, and the system moved to the next process. Finally, the algorithm produced a safe sequence, which ensures that all processes can complete without entering a deadlock.

Although the Banker's Algorithm provides strong deadlock avoidance, it requires processes to declare maximum resource demands beforehand, which may not always be practical. Nonetheless, this experiment clearly demonstrates how operating systems prevent deadlock using resource-allocation strategies.